

Communication

Date	July 9, 2026
Team ID	SWTID-2026-1262
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	1 Mark

Communication Plan:

S.No	Communication Type	Frequency	Channel / Tool	Participants	Purpose
1	Team Standup	Daily	Slack / WhatsApp Status Sync	Hari Charan	Review daily code progression across the frontend Bootstrap modules and backend Scikit-Learn evaluation scripts.
2	Progress Update	Weekly	GitHub Commit History / Trello	Hari Charan	Document successful implementations of core operational user scenarios (Scenario 1, 2, and 3) and evaluate training model metrics.
3	Issue / Bug Discussion	As Needed	VS Code Live Share / Discord	Hari Charan	Diagnose runtime compilation issues, unexpected indentation bugs, or Anaconda virtual environment library mismatches.
4	Stakeholder Review	Bi-Weekly	Microsoft Teams / Zoom	Hari Charan	Demonstrate working browser interface previews, input boundary safety behaviors, and pickled model serialization outputs.
5	Final Demo Rehearsal	Once	Local Desktop Walkthrough Recording	Hari Charan	Run terminal sanity checks to ensure the application starts flawlessly from a fresh requirements.txt environment boot.

Communication Challenges & Resolutions:

S.No	Challenge Faced	Resolution / Action Taken
1	Workspace Execution Sync Errors: Code Runner executing detached blocks of code resulting in sudden indentation anomalies (IndentationError) over loop states.	Implemented explicit terminal instructions bypassing scratchpad script executions. Standardized local development runs strictly through direct conda prompt commands (python train_model.py).
2	Strict Diagnostic Type Overload Alerts: Static type checkers (Pylance) triggering validation warnings when mapping model arrays inside shorthand dictionary tracking functions.	Refactored the core training model pipeline comparison logic to utilize standard procedural tracking loops, resolving structural warnings cleanly.
3	Input Vector Out-of-Range Risk: Handling extreme or corrupted user data entries that could bypass model inputs and cause sudden server runtime exceptions.	Formulated strict, readable validation boundaries inside the Flask gateway route (app.py), throwing structured HTTP 400 bad request alerts whenever an out-of-bounds float array is entered.